

CircuitMentor

Embedded Systems AI Tutor

Version 1.0 | Project Manual | April 2026

Built by
Rajas L

Hardware
NVIDIA Jetson AGX Orin 64GB

PERSONAL PROJECT — ALL INTELLECTUAL PROPERTY BELONGS TO THE AUTHOR

1. What is CircuitMentor?

CircuitMentor is a domain-specific Small Language Model (SLM) — a fine-tuned AI that acts as an expert teaching assistant for junior ECE students. It specializes exclusively in embedded systems: Arduino, ESP32, Raspberry Pi Pico, ATmega, STM32, MicroPython, Embedded C, C++, IoT protocols, and FreeRTOS.

Unlike general-purpose AI tools like ChatGPT, CircuitMentor runs entirely on-device on a NVIDIA Jetson AGX Orin edge device — with no cloud dependency, no API calls, and no internet required during use. The dataset was curated, the model was fine-tuned using LoRA, and the full deployment stack was designed and built independently from scratch.

1.1 Comparison with Cloud AI

Feature	CircuitMentor	ChatGPT / Cloud LLM
Runs on-device	Yes — Jetson AGX Orin	No — remote datacenter
Works offline	Yes — no internet needed	No — requires internet
Domain specialization	Embedded systems only	General purpose
Data privacy	Nothing leaves device	Data sent to cloud
Cost per query	Zero after hardware	Per-token billing
Model ownership	Full — weights owned	Closed model

2. Technical Specifications

2.1 Hardware

Component	Specification
Device	NVIDIA Jetson AGX Orin
Unified Memory	64 GB (CPU + GPU shared)
CUDA Version	12.6
JetPack	6.1 (R36.4.7)
Storage	465 GB NVMe
OS	Ubuntu 22.04

2.2 Software Stack

Component	Version / Detail
Base Model	Meta Llama-3.1-8B-Instruct
Fine-Tuning	LoRA (Low-Rank Adaptation) via PEFT
Precision	bf16 — CUDA 12.6 safe, no bitsandbytes
Python	3.10.12
PyTorch	2.5.0a0 nv24.8
Transformers	4.44.0 (HuggingFace)
Backend	FastAPI + asyncio queue
Frontend	Streamlit
Public tunnel	ngrok

2.3 Training Configuration

Parameter	Value	Reason
LoRA rank (r)	16	Standard for 15k dataset
LoRA alpha	32	2x r — standard scaling
Trainable params	41.9M / 8.07B (0.52%)	LoRA efficiency
Epochs	3	Best convergence for dataset size
Batch size	1 + grad_accum=16	Jetson memory constraint
Learning rate	2e-4 (cosine decay)	Standard LoRA LR
Memory cap	50 GB	Leaves 14GB for OS
NEFTune noise	alpha = 5.0	Boosts instruction following
Final train loss	0.1855	Excellent — beats expected 0.50
Training time	~40 hours total	Including 3 power cuts, resumed each time
Total steps	2000 / 2529	Completed and merged cleanly

3. Dataset Construction

The dataset was built entirely from scratch using a custom two-phase pipeline (build_dataset.py). No pre-existing embedded systems dataset was used directly. All sourcing, filtering, scoring, and formatting was implemented independently.

3.1 Phase 1 — Raw Collection (54,152 examples)

Source	Scanned	Passed Filter
Glaive-Code-Assistant	30,000	6,647
Magocoder-Evol-110k	40,000	5,300
WizardLM-Evol-196k	20,000	1,850
OpenHermes	20,000	1,125
Code-Instructions-122k	40,000	41
Alpaca-Code-18k	18,000	13
CodeAlpaca-20k	20,000	12
Synthetic Seeds (hand-crafted)	—	12
Total	188,000+	15,000 selected

3.2 Final Dataset Stats

Stat	Value
Train split	13,500 examples (90%)
Val split	1,500 examples (10%)
With code blocks	81% of examples
Format	Alpaca instruction/input/output
Decontamination	MD5 hash deduplication — clean

4. System Architecture

CircuitMentor uses a three-layer architecture: a FastAPI inference backend, a Streamlit frontend UI, and an ngrok tunnel for public access. All three run on the Jetson Orin simultaneously.

4.1 Request Flow

```
User browser (anywhere in the world)
|
|  https://holy-monoclinic-jule.ngrok-free.dev
v
ngrok tunnel (cloud relay - free tier)
|
v
Streamlit Frontend - port 8501 (frontend.py)
|  HTTP POST /chat
v
FastAPI Backend - port 8000 (backend.py)
|  asyncio queue - ONE request at a time
v
Llama-3.1-8B Fine-Tuned Model (bf16, GPU)
/mnt/circuitmentor/experiments/llama3_tutor_v1/final_model
|
v
Response returned to user (~3-8 seconds)
```

4.2 File Structure

```
/mnt/circuitmentor/
├── app/
│   ├── backend.py      - FastAPI, model inference, session management
│   ├── frontend.py     - Streamlit UI, easter eggs, responsive design
│   ├── launch.py       - Single command launcher
│   └── train_tutor.py  - LoRA training script
├── datasets/
│   ├── train.jsonl     - 13,500 training examples
│   └── val.jsonl       - 1,500 validation examples
├── experiments/llama3_tutor_v1/
│   ├── final_model/    - Merged model (~15GB safetensors)
│   └── final_lora_adapter/ - LoRA weights only (~161MB)
├── sessions/          - Live UUID session folders (auto-wiped on exit)
├── logs/              - Server access logs only
├── start.sh           - Start everything (one command)
├── stop.sh            - Stop everything
└── config.json        - Ports, max users, admin credentials hash
```

4.3 Session & Queue Design

- Maximum 10 simultaneous users — 11th user sees friendly full message
- Each user gets a unique UUID session folder under /mnt/circuitmentor/sessions/
- Same username cannot login twice simultaneously
- Maximum 200 messages per session
- asyncio FIFO queue — model processes one request at a time
- 90 second queue timeout — user gets friendly message if server busy
- On exit: user prompted to download chat as .txt before deletion
- Session folder completely wiped on exit — zero data retained on server

5. Features

5.1 Core AI Capabilities

- Embedded C and C++ code generation with comments and explanations
- MicroPython and CircuitPython for ESP32, Raspberry Pi Pico
- Arduino sketches with full explanation of every function
- Protocol deep dives: I2C, SPI, UART, MQTT, BLE, LoRa
- FreeRTOS task creation and scheduling on ESP32
- Hardware debugging: watchdog timer, power issues, serial monitor
- Sensor interfacing: DS18B20, DHT11/22, BME280, MPU6050, NeoPixel
- IoT connectivity: WiFi, MQTT broker, OTA updates

5.2 UI and Access Features

- Responsive design — works on desktop, tablet, and mobile browser
- Dark professional theme with syntax-highlighted code blocks
- 8 one-click example prompts covering all major topics
- Temperature and max token generation controls
- Response time displayed per answer (e.g. answered in 4.2s)
- Download full chat as .txt before session ends
- Admin dashboard: active users, RAM/GPU stats, kick and clear controls
- Public URL via ngrok — accessible from anywhere in the world

5.3 Easter Eggs

Trigger	Effect
Click logo 4 times	Credits overlay: Rajas L, Pranav C Sir, Nikhil Z Sir
Konami code (keyboard arrows + BA)	Binary matrix rain animation for 2.5 seconds
Type: who made you	Credits card appears in chat
Type: hello world	Hello World in 5 embedded languages
Type: sudo anything	sudo apt install patience joke
Ask about Verilog or FPGA	That is my cousin's department — VLSI lab
Idle 5 minutes	Popup: The circuit won't build itself
Idle 10 minutes	Warning: Session closing soon

6. How to Use CircuitMentor

6.1 Starting the Server

SSH into the Jetson Orin, then run:

```
bash /mnt/circuitmentor/start.sh
```

Wait 40 seconds. The model loads into GPU memory and the ngrok tunnel opens. The permanent public URL is:

```
https://holy-monoclinic-jule.ngrok-free.dev
```

6.2 Stopping the Server

```
bash /mnt/circuitmentor/stop.sh
```

6.3 Check if Running

```
ps aux | grep -E 'launch|uvicorn|streamlit|ngrok' | grep -v grep
```

6.4 User Guide

- Open <https://holy-monoclinic-jule.ngrok-free.dev> in any browser
- Enter your name on the join screen and click Join CircuitMentor
- Type any embedded systems question and click Send
- Use sidebar example prompts for quick topic exploration
- Adjust Temperature (lower = focused) and Max Tokens in sidebar
- Click Exit Session when done — save chat as .txt if needed
- All session data is permanently deleted on exit

6.5 Admin Access

- Click Admin Login on the join screen
- Username: SungjinHo
- Password: circuitmentor_admin_2026
- Dashboard: active sessions, usernames, message counts, RAM/GPU usage
- Kick button force-removes any user session
- Clear All Sessions button wipes all sessions at once

7. Build Timeline

This project was built entirely as a personal initiative in approximately 2 weeks, while the primary VLSI M4 project was simultaneously running on the same hardware.

Date	Activity
Mar 27	VLSI M3 milestone completed. Concept for CircuitMentor formed.
Mar 28-30	Designed dataset architecture. Wrote build_dataset.py with 2-phase pipeline.
Mar 30	Dataset complete: 54,152 raw collected → 15,000 cleaned. 81% code coverage.
Apr 1	Llama-3.1-8B downloaded (16GB). Model integrity verified.
Apr 1-3	Designed full system: FastAPI backend, Streamlit UI, ngrok, session management, easter eggs.
Apr 4 (AM)	VLSI M4 training completed. Orin freed. SCP model + dataset to Orin.
Apr 4 14:39	CircuitMentor training launched. Llama-3.1-8B + LoRA r=16, 3 epochs.
Apr 4 18:15	Training crashed at step 100 — f-string logging bug. Fixed and restarted.
Apr 5 (AM)	Power cut. Root partition 100% full (student files). Cleared 8GB. Resumed from checkpoint-1400.
Apr 6 00:10	Training COMPLETE. Final loss: 0.1855. LoRA adapter + merged model saved.
Apr 6 06:21	Inference test passed. Arduino LED blink code — correct and well-explained.
Apr 6 06:48	CircuitMentor LIVE. ngrok tunnel active. Public URL printed.
Apr 6 07:07	First full test session completed successfully. Chat export verified.

7.1 Obstacles Overcome

- 3 power cuts survived — checkpoint-based auto-resume system used each time
- Root partition 100% full — identified and cleared 8GB of leftover student files
- bitsandbytes incompatible with CUDA 12.6 — solved with bf16 + standard LoRA
- f-string logging bug at step 100 — fixed and restarted without losing progress
- Incomplete model merge (power cut during save) — re-merged from LoRA adapter
- /tmp filesystem full — redirected TMPDIR to /mnt/circuitmentor/tmp
- Meta AI identity bleed in responses — fixed via hardened system prompt + EOS stop tokens

8. Key Technical Concepts

8.1 What is LoRA?

LoRA (Low-Rank Adaptation) fine-tunes a tiny fraction of model parameters instead of all 8 billion. Instead of updating the full weight matrix W , it trains two small matrices A and B whose product approximates the change.

```
Full weight matrix W:  [4096 x 4096] = 16.7M parameters
LoRA approximation:    A [4096 x 16] + B [16 x 4096] = 131K parameters
Savings: 128x fewer parameters per layer

CircuitMentor: 41.9M trained / 8,072M total = 0.52% of all parameters
```

8.2 Why bf16 and not int4/int8?

bitsandbytes (required for 4-bit and 8-bit quantization) is incompatible with CUDA 12.6 on the ARM64 Jetson architecture. bf16 uses the same 16-bit memory footprint as fp16 but with better numerical stability — eliminating the NaN gradient explosions that fp16 causes on Jetson. This was discovered and validated in the prior VLSI M3 milestone.

8.3 Final Loss Analysis

Metric	Value	Assessment
Final train loss	0.1855	Excellent — far better than expected 0.50
Eval loss	0.8788	Healthy — model generalizes well
VLSI M4 final loss	0.22	CircuitMentor beats this at 0.1855
Overfitting risk	Low	Gap between train and eval is acceptable

9. Script Reference

File	Purpose
build_dataset.py	Two-phase dataset pipeline. Phase 1: streams and filters 188k+ rows from 8 HuggingFace sources. Phase 2: scores and cleans to 15k examples.
train_tutor.py	LoRA fine-tuning script. bf16, gradient checkpointing, NEFTune noise, checkpoint every 200 steps, auto-resume support, emergency checkpoint on interrupt.
backend.py	FastAPI server. Loads model once at startup. asyncio queue (1 inference at a time). UUID session management. Admin auth with SHA-256 hashing. All REST endpoints.
frontend.py	Streamlit UI. Fully responsive (desktop/tablet/mobile). Dark theme. All easter eggs. Chat export. Admin dashboard. CircuitMentor glasses logo.
launch.py	Single command launcher. Pre-flight checks (model, deps, dirs). Starts backend + frontend + ngrok. Monitors for crashes. Clean shutdown on Ctrl+C.
start.sh	Shell wrapper using nohup. One command brings everything live in background. Prints URL after 40 seconds.
stop.sh	Kills all CircuitMentor processes cleanly with pkill.

10. Credits & Ownership

CircuitMentor v1.0
Built by Rajas L
April 2026 | Personal Project

10.1 Third-Party Credits

Component	Credit
Base Model	Meta Llama-3.1-8B-Instruct — Meta AI (meta-llama license)
Fine-Tuning Framework	HuggingFace TRL + PEFT (Apache 2.0)
Dataset Sources	Glaive, Magicoder, WizardLM, OpenHermes (research-permissive licenses)
UI Framework	Streamlit (Apache 2.0)
Backend	FastAPI (MIT License)
Public Tunnel	ngrok (free tier)
Hardware	NVIDIA Jetson AGX Orin

All code, architecture, dataset pipeline, training configuration, deployment stack, and UI were designed and implemented independently by Rajas L.